

برنامه کارآموزی آزمایشگاه رباتیک

تمرین هفته دوم: اجرای وظایف برنامه‌ریزی شده روی TurtleBot3 – قسمت اول

مقدمه

در تمرین هفته اول، یک سیستم ساده برای مدیریت و اجرای وظایف ربات طراحی کردید. این سیستم شامل چند Node بود که از طریق ROS 2 با یکدیگر ارتباط داشتند و عملیات ربات را به صورت شبیه‌سازی شده اجرا می‌کردند.

در این تمرین قرار است همان سیستم را یک مرحله به یک سیستم رباتیک واقعی‌تر نزدیک کنیم. در این مرحله، به جای اینکه اجرای عملیات حرکت با گذشت زمان شبیه‌سازی شود، ربات TurtleBot3 Waffle در محیط Gazebo واقعاً حرکت خواهد کرد.

همچنین به جای اینکه ترتیب عملیات به صورت دستی در Task Controller مشخص شود، از یک Planner برای تولید برنامه‌ی اجرای وظایف استفاده خواهد شد. برای این منظور، از نسخه موجود PDDLStream که در آزمایشگاه توسعه داده شده است استفاده خواهید کرد.

هدف اصلی تمرین، آشنایی با نحوه اتصال یک Task Planner به سیستم اجرایی ربات و درک ارتباط میان برنامه‌ریزی سطح بالا و کنترل ربات است.

سناریوی مسئله

فرض کنید یک ربات در محیطی مشابه محیط‌های مورد استفاده در مسابقات RoboCup@Work قرار دارد. ربات باید بتواند بین مکان‌های مختلف محیط حرکت کند. برای مثال، ممکن است یک وظیفه شامل موارد زیر باشد:

MOVE_TO_WORKSTATION
PICK_OBJECT
MOVE_TO_SHELF
PLACE_OBJECT

در این تمرین هنوز بازوی ربات در اختیار شما نیست؛ بنابراین عملیات‌های PICK و PLACE فقط به صورت فرضی و شبیه‌سازی شده اجرا می‌شوند و نسبت به تمرین هفته قبل تغییری ندارند. اما عملیات MOVE_TO باید به صورت واقعی روی TurtleBot3 شبیه‌سازی شده انجام شود. در هفته‌های بعد، عملیات Place و Pick نیز با استفاده از بازوی ربات توسعه داده خواهند شد.

در معماری نهایی، Planner وظیفه تصمیم‌گیری درباره‌ی اینکه چه کاری باید انجام شود را بر عهده دارد و بخش کنترل ربات وظیفه دارد آن کار را روی ربات اجرا کند. بنابراین Planner نباید مستقیماً با cmd_vel کار کند و بخش اجرایی سیستم باید این عملیات را به یک حرکت واقعی TurtleBot3 تبدیل کند.

معماری سیستم

برنامه‌ریز PDDLStream:

PDDLStream وظیفه تولید یک برنامه قابل اجرا برای مسئله را بر عهده دارد. Planner باید بتواند با توجه به مسئله، مجموعه‌ای از actionها را تولید کند. در این تمرین از نسخه موجود PDDLStream در آزمایشگاه استفاده می‌کنید و نیازی به تغییر الگوریتم یا توسعه قابلیت‌های جدید برای PDDLStream وجود ندارد. همچنین نیازی به تبدیل نسخه فعلی PDDLStream از Continuous 2D به محیط سه بعدی نیست. در این تمرین PDDLStream را به عنوان یک Planner در نظر بگیرید که خروجی آن شامل اطلاعات مورد نیاز برای اجرای وظایف است.

از طریق این لینک [میتوانید به کدهای برنامه‌ریز دسترسی داشته باشید](#). با دنبال کردن دستورالعمل‌های موجود در README پروژه میتوانید برنامه‌ریز را روی سیستم خودتان راه اندازی کنید. در این پروژه صرفاً به domain دوم (Continuous2D) نیاز خواهید داشت که سناریوهای مختلف آن را میتوانید با روش گفته شده در فایل README پروژه اجرا نمایید. بعد از اجرا، در صورتی که برنامه‌ریز توانسته باشد برنامه مناسبی پیدا کند خروجی شامل متریک‌های اندازه‌گیری شده در طول اجرا و اکشن‌های لازم برای رسیدن به هدف چاپ میشود و قابل مشاهده است. در این هفته شما صرفاً به خروجی‌های اکشن move_base نیاز دارید.

در شکل صفحه بعد یک نمونه از اکشن‌های خروجی داده شده توسط الگوریتم را مشاهده می‌کنید. در خروجی مربوط به اکشن move_base سه بخش اصلی وجود دارد: **نقطه شروع حرکت**، **trajectory** و **مسیری که باید طی بشود تا ربات به مقصد حرکت برسد** و **مختصات نقطه هدف یا مقصد**. سیستم اجرایی باید این trajectory را دریافت کرده و با استفاده از یک Trajectory Follower، حرکت TurtleBot3 را در Gazebo اجرا کند.

```
Summary: {complexity: 4, cost: 122.776, evaluations: 315, iterations: 5, length: 2, run_time: 3.766, sample_time: 0.414, search_time: 3.352, skeletons: 1, solutions: 1, solved: True, timeout: False}

Solved: True
Cost: 122.776
Length: 20
Deferred: 0
Evaluations: 2
1) move_base robot0 BaseConf(x=0.0, y=1.0, theta=0.0) BaseTraj waypoints=[BaseConf(x=0.0, y=1.0, theta=0.0), BaseConf(x=np.float64(0.04230640372070172), y=np.float64(0.856116234600043), theta=np.float64(-1.284276807160000)), BaseConf(x=np.float64(0.14058796833893017), y=np.float64(0.7427212003746468), theta=np.float64(-0.8571299389525839)), BaseConf(x=np.float64(0.2892439669008512), y=np.float64(0.7226863783999414), theta=np.float64(-0.1339658310531576)), BaseConf(x=np.float64(0.4047634877483594), y=np.float64(0.6270033584687011), theta=np.float64(-0.6917511844480639)), BaseConf(x=np.float64(0.528055416698181), y=np.float64(0.5415685865322575), theta=np.float64(-0.6059767043753116)), BaseConf(x=np.float64(0.46894268211703116), y=np.float64(0.4037074803045091), theta=np.float64(-1.9758682859773393)), BaseConf(x=np.float64(0.3553603780010942), y=np.float64(0.3057326866496368), theta=np.float64(-2.4298355413343913)), BaseConf(x=np.float64(0.3201496004262869), y=np.float64(0.15992390384634528), theta=np.float64(-1.8077458870906997)), BaseConf(x=np.float64(0.32452625229203386), y=np.float64(0.009987767713503515), theta=np.float64(-1.5416145060969744)), BaseConf(x=np.float64(0.3719710177042821), y=np.float64(-0.13231119305212094), theta=np.float64(-1.2489712786842286)), BaseConf(x=np.float64(0.4883094704327492), y=np.float64(-0.22699680133947486), theta=np.float64(-0.6831475942761882)), BaseConf(x=np.float64(0.6280441050660538), y=np.float64(-0.28153332014932264), theta=np.float64(-0.37210458526823476)), BaseConf(x=np.float64(0.76642442097382), y=np.float64(-0.3394201767686522), theta=np.float64(-0.39619660728717254)), BaseConf(x=np.float64(0.8518018078057266), y=np.float64(-0.4627518507934338), theta=np.float64(-0.9652849882873223)), BaseConf(x=np.float64(0.9776642705655197), y=np.float64(-0.3811513577309549), theta=np.float64(0.5752007970804321)), BaseConf(x=np.float64(1.0709658183022055), y=np.float64(0.49860271442783993), theta=np.float64(-0.8994888070287892)), BaseConf(x=np.float64(1.1642673660388914), y=np.float64(-0.616054071124725), theta=np.float64(-0.8994888070287891)), BaseConf(x=np.float64(1.311162074140378), y=np.float64(-0.6464176139315025), theta=np.float64(-0.20383214360548063)), BaseConf(x=np.float64(1.4437207830178842), y=np.float64(-0.5762165548174938), theta=np.float64(0.4870342694206134)), BaseConf(x=np.float64(1.515028328434036), y=np.float64(-0.7081833459686142), theta=np.float64(-1.0753962865976112)), BaseConf(x=np.float64(1.520883914439423), y=np.float64(-0.8580690094342489), theta=np.float64(-1.5317491651055266)), BaseConf(x=np.float64(1.52673950044481), y=np.float64(-1.0079546728998836), theta=np.float64(-1.5317491651055266)), BaseConf(x=np.float64(1.5293852342868517), y=np.float64(-1.0756776252915097), theta=-2.824925795286977)), BaseConf(x=np.float64(1.5293852342868517), y=np.float64(-1.0756776252915097), theta=-2.824925795286977))
2) pick_and_stow robot0 s40_40_2 (1.15, -1.2) Grasp(obj='s40_40_2', dx=0.0, dy=0.0, dtheta=0.0) BaseConf(x=np.float64(1.5293852342868517), y=np.float64(-1.0756776252915097), theta=-2.824925795286977) ArmConf(joints=(0.0, 0.0, 0.0)) ArmConf(joints=(np.float64(-0.868517), y=np.float64(-1.0756776252915097), theta=-2.824925795286977))
```

Task Controller

همان Task Controller اصلی تمرین هفته اول است و باید تا حد امکان از ساختار قبلی استفاده شود.

وظیفه آن دریافت plan، مدیریت ترتیب اجرای actionها، ارسال action فعلی به بخش اجرایی، دریافت نتیجه اجرای action، ارسال action بعدی پس از موفقیت action قبلی و مدیریت خطا است.

Robot Controller

در تمرین هفته اول، اجرای عملیات ربات توسط Robot Simulator با استفاده از زمان و توابع شبیه‌سازی می‌شد. در این تمرین، بخش مربوط به حرکت باید با TurtleBot3 جایگزین شود. Robot Controller باید بتواند یک action از نوع MOVE_TO دریافت کند و ربات را به موقعیت مورد نظر در محیط Gazebo منتقل کند. به عبارتی دیگر Robot Controller باید با استفاده از اطلاعات موقعیت فعلی ربات و ارسال فرمان مناسب به cmd_vel، ربات را به مقصد برساند.

ساختار تحویل پروژه

پروژه باید در یک ROS 2 package مناسب قرار گیرد.

در کنار کد، یک فایل README.md قرار دهید و موارد زیر را در آن توضیح دهید:

- معرفی کوتاه پروژه
- معماری سیستم
- نقش PDDLStream
- Nodeهای استفاده شده
- Topicها و Serviceهای استفاده شده
- نحوه اجرای PDDLStream
- نحوه اجرای TurtleBot3 در Gazebo
- نحوه اجرای Nodeهای پروژه
- نحوه اجرای سناریو
- نحوه تبدیل actionهای MOVE_TO به حرکت ربات
- نحوه مدیریت خطا
- نمونه‌ای از خروجی اجرای سیستم
- مشکلات مهمی که در طول پیاده‌سازی با آنها مواجه شدید و نحوه حل آنها

همچنین حداقل یک تصویر یا نمودار از معماری سیستم و یک نمونه خروجی موفق اجرای task در README قرار دهید.

نکته پایانی

در این تمرین هدف اصلی، پیاده‌سازی یک سیستم پیچیده نیست. هدف این است که مفاهیم پایه ROS 2 را در قالب یک مسئله کوچک اما نزدیک به ساختار سیستم‌های واقعی رباتیک به کار بگیرید.

در مراحل بعدی، اجزای شبیه‌سازی شده این پروژه می‌توانند با اجزای واقعی ربات، سنسورها، کنترلرها و سیستم‌های تصمیم‌گیری جایگزین شوند. بنابراین در طراحی Nodeها و interfaceهای ارتباطی، سعی کنید ساختاری منظم و قابل توسعه ایجاد کنید.

